Abstract

This individual C++17/Python project evaluates six sequential sorting implementations, OpenMP Merge Sort, three array searches, Skip List and four B-tree orders, plus a simple learned rank index. The retained CPU experiments and new held-out learned/10M/100M studies provide descriptive evidence. The new scale extension completed 600 measured sorts and 120 warm-ups, all correct. The learned study completed 480 measured query batches and 240 measured builds. This release is CPU-only: GPU/CUDA/CUB execution and comparisons are deferred, not completion requirements. No CPU-versus-GPU performance result is claimed. No universal performance ranking or significance claim is made.

Introduction and research questions

How do input order and duplicate frequency affect concrete sort implementations? How do query outcome and location affect array and ordered-index lookup? How does OpenMP scaling compare with sequential Merge Sort? When does a rank model require correction or fallback? How do separate key-comparison and process-memory observations inform timing? The project addresses these questions with deterministic inputs, full oracle checking and retained raw data.

Related work and implementation identities

PDQSort is Orson Peters' upstream implementation at commit b1ef26a55cdb60d236a5cb199c4234c704f46726 (Zlib). Timsort is cpp-TimSort at99b612d9826e8387a30008f2b21be206181ad890 (MIT). std_sort is this toolchain's standard-library sort, not renamed PDQSort or Timsort. QuickSort has a depth budget and heapsort fallback. Merge Sort is stable and uses a temporary vector. Signed Radix Sort uses four stable byte passes and a signed-order bit mapping. OpenMP tasks retain the sequential baseline. See docs/ALGORITHMS.md and vendor UPSTREAM.json files for licenses/hashes. Deferred GPU code is excluded from this release's result inputs. The learned endpoint model and AlphaDev research review are documented in docs/LEARNED_INDEX_PROTOCOL_v1.md and docs/ALPHADEV_STUDY.md.

Methodology and environment

Signed int32 generator-v1 uses a fresh mt19937_64(seed); distributions and rejection sampling are frozen in EXPERIMENT_PROTOCOL.md. All corresponding sorting calls use identical original inputs. Sorting uses steady_clock nanoseconds, excludes generation, input restoration, oracle work and serialization, and checks complete output equality after timing. Primary timing contains no comparison instrumentation. One warm-up and five repetitions are used. New CPU runs use Linux Pop!_OS kernel6.18.7, GCC13.3.0, CMake3.28.3, -O3 -DNDEBUG, Intel i5-11400H (6 physical/12 logical cores), and approximately16.48GB RAM. Governors report powersave; background load, caches, temperature and frequency were not controlled. Agent-launched benchmarks and builds were serialized; external background work was uncontrolled. Historical studies retain their own metadata and must not be treated as the same session. Study protocols and exact commands reside in the audit.

Results: retained 1M sorting study

Historical evidence:15 dataset configurations;450 measured sorts and90 warm-ups; six sorters, five distributions and three seeds. These observations show workload-dependent differences between concrete implementations, not a universal ordering.

Full tables and sources: FINAL_REPORT.md / INPUTS_SHA256.json

Results: CPU scale extension

10M:15 datasets across five distributions and three seeds;450 measured/90 warm-up sorts. 100M:five datasets across five distributions and seed1729;150 measured/30 warm-up sorts. All six sorters completed every configuration with full equality to the oracle. The policy estimate was160MB array payload at10M and1.6GB at100M, with separate512MiB/2048MiB caps and more than3GiB RAM safety headroom. These planning quantities are not measured RSS. The100M command-level lifetime RSS is retained in the /usr/bin/time log; it cannot isolate a single sorter.

Results: search and ordered structures

The retained full CPU pipeline includes 7020 measured search batches and 13200 measured structure operations. Construction and lookup remain separate. Binary/exponential search return the first equal array index; structures implement ordered maps with unique keys and first-inserted payloads. Mixed query batches use exact50% hit/miss ratios. The following is a representative retained uniform group. Batch time/query count is a normalized average, not a directly measured individual-query latency or maximum.

Results: learned rank index

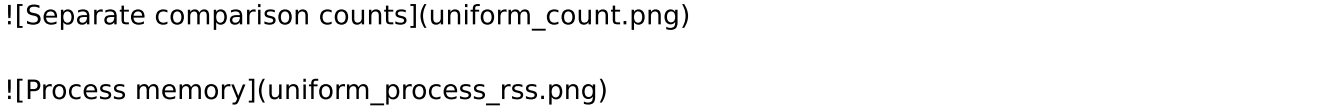
The fixed endpoint affine model uses radius32, certified local lower-bound correction and a full binary-search fallback. The evaluation has48 workload configurations (1K/10K/100K/1M, four distributions, seeds42/2718/314159) and1000 queries/configuration,500 hits/500 misses. These contain24 distinct sorted input vectors: uniform, reverse and nearly sorted collapse to identical contents after sorting for a given size/seed. Workload labels are kept separate and are not pooled as independent datasets. Evaluation seeds are held out from performance tuning; some also appear in correctness tests, which did not tune the model. The indexed vector itself supplies model endpoints; this is not unseen-key neural generalization. 480 measured lookup batches,240 measured builds,144 warm-ups and48000 untimed diagnostic queries completed with no failures. Build includes the owned copy and validation. Query batches exclude build and diagnostics. summary.json retains fallback rates, correction distance, separate comparisons and logical memory. The model does not guarantee O(1) lookup; duplicate/skewed data can force O(logN) fallback.

Results: OpenMP scaling

The retained OpenMP scaling study completed1800 measured sorts and360 warm-ups across requested1/2/4/8-thread configurations at32768/100000/1000000 keys. Its primary speedup baseline is sequential Merge Sort; parallel one-thread timing is separately identified. Speedup is T_sequential/T_parallel and efficiency is speedup/observed team size; values below1 are retained. Ratios are calculated within datasets before equal weighting distinct input fingerprints. The uniform1M table uses three seed datasets, each with five timings per implementation. Team size does not establish task participation, physical-core concurrency or race freedom.

Results: comparison counts and process memory

240 separate sort_metrics executions provide exact key-comparison counts for five adapters and process RSS. Radix counting is unsupported. Counts have consistent executed-key-predicate semantics but do not model moves, cache misses or total instructions. RSS includes input/runtime/allocator/instrumentation and is not isolated auxiliary space. Each size/distribution/algorithm has three seed observations, not repeated RSS samples. Median and min–max across these observations describe different inputs, not independent process-repeatability estimates.



Release scope and deferred work

This is a complete CPU-only release for the declared studies. GPU execution, CUDA/CUB comparisons and the frozen CPU/GPU full study are outside the release acceptance criteria. Their source, pins, protocols, matrices and failed/partial validation attempts are preserved in place and indexed by docs/DEFERRED_GPU.md. Historical GPU pilots are separate earlier experiments, not observations from the unexecuted full protocol. No GPU numbers enter the report tables or plots, and no CPU-versus-GPU conclusion is made. The independent CPU/GPU approval gate remains enforced and unmodified; it is not a prerequisite for local CPU analysis or correctness. Individual-query maxima, isolated structure allocations and independent-machine replication remain documented limitations, not fabricated deliverables.

Statistical interpretation and threats to validity

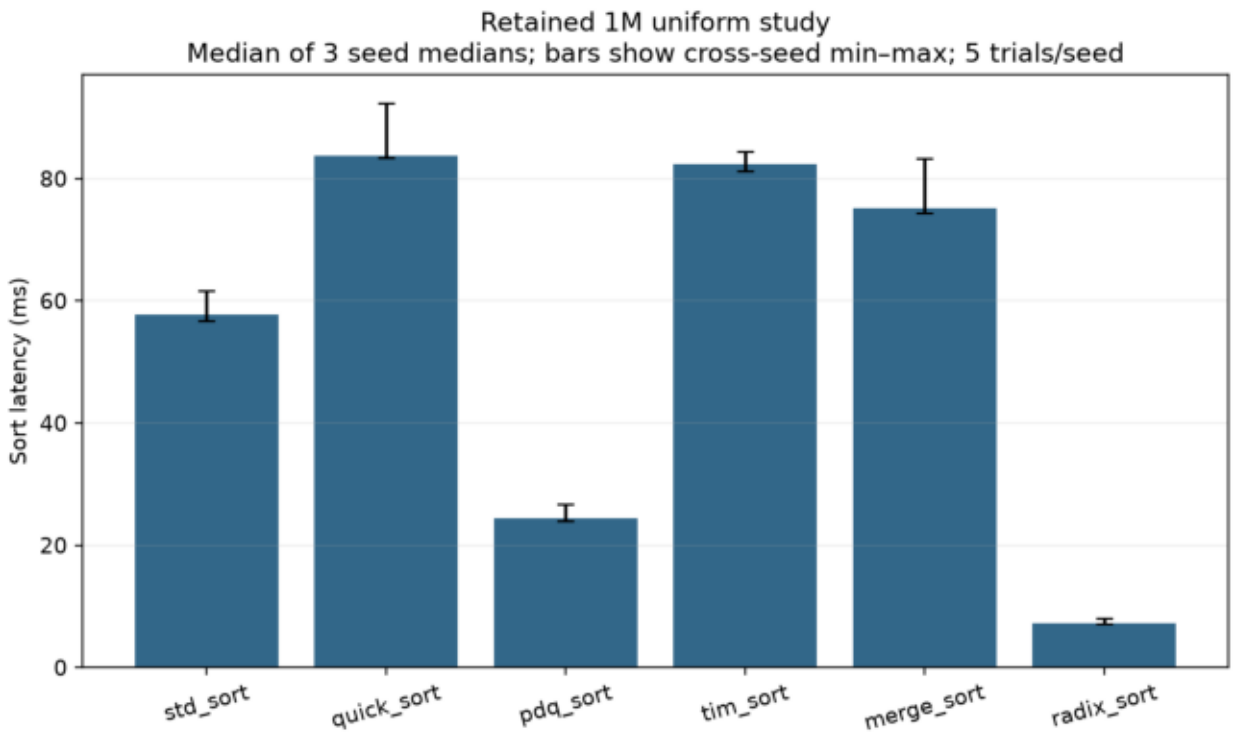
Repeated timings are first summarized within each dataset using median, mean, sample SD, min and max. Cross-seed summaries equally weight dataset medians; identical fingerprints are collapsed where the family analyzer specifies it. No outliers are discarded and no confidence interval or significance claim is inferred from three seeds. 100M uses one seed, so only within-dataset variability exists. Shared runtime scheduling, SMT, allocator/cache history, frequency/thermal behavior and background load can confound observations. Key-only tests cannot empirically distinguish the order of equal payloads; stability follows the documented merge strategy and existing generic stability tests. Search individual-query maxima are not collected because the batch engine does not measure them; normalized batch maxima cannot substitute. Structure comparison/allocation metrics and a learned sorter selector remain unsupported/optional. ThreadSanitizer/OpenMP limitations in earlier audits mean ordinary passing tests are not a proof of race freedom.

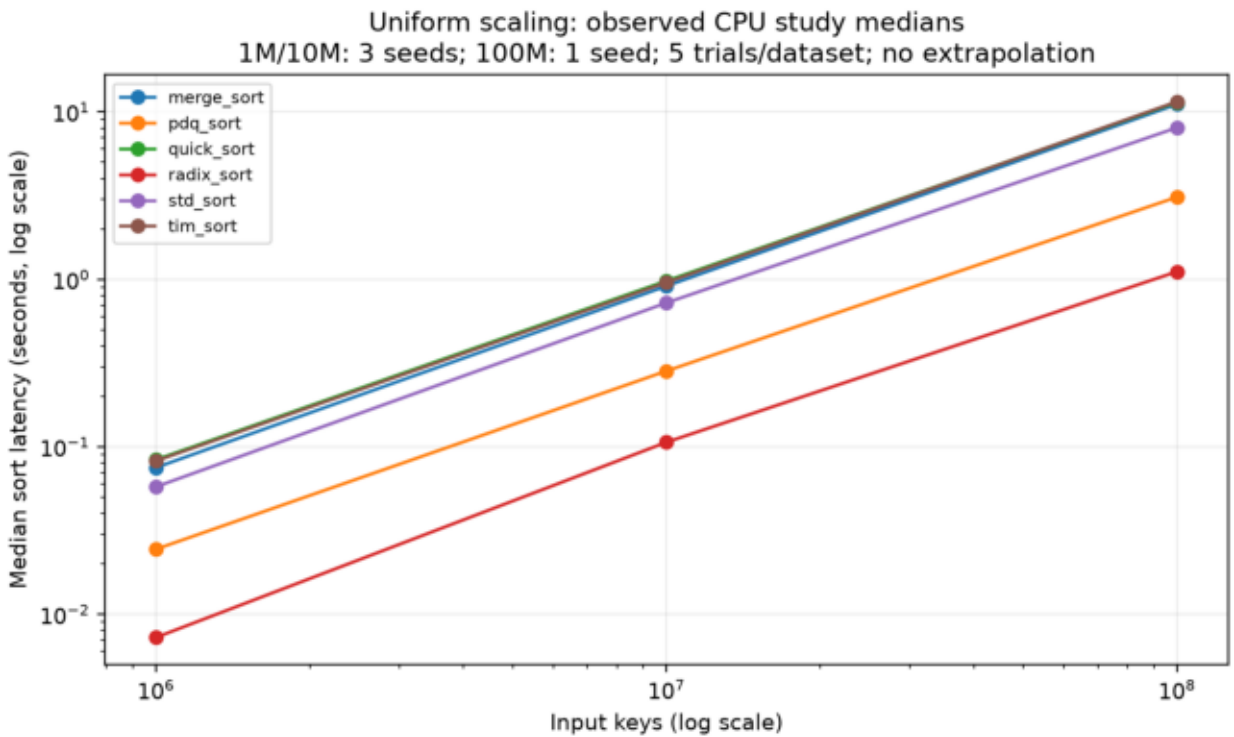
Conclusion, reproducibility and references

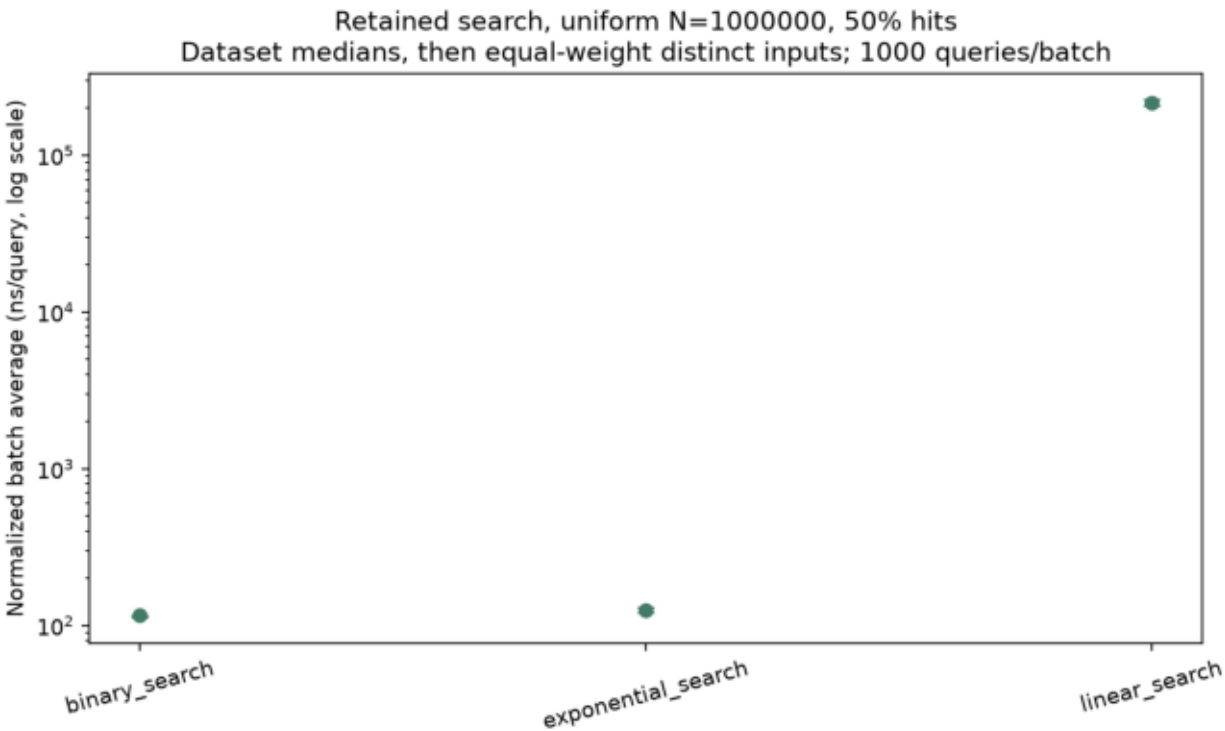
The validated CPU implementation, held-out model evaluation and completed100M feasibility extension support workload-specific analysis. The release conclusions concern CPU implementations only. GPU comparisons are deferred and do not prevent completion of this CPU-only scope. AlphaDev is reviewed from primary sources; no reinforcement-learning training or assembly discovery was reproduced.

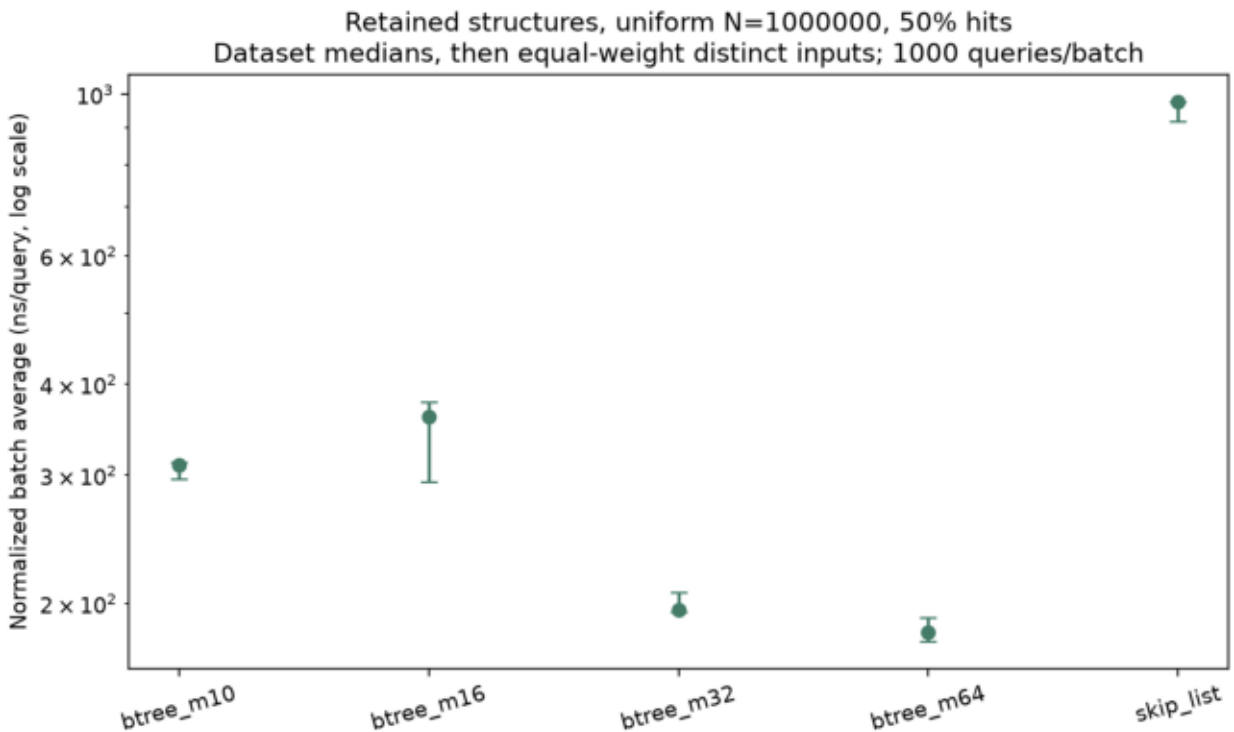
Regenerate this report with `python3 report/generate.py --evidence results/raw/autonomous_completion_20261011T095554571762Z --learned results/processed/learned_rank_20261011T095554571762Z --metrics results/processed/count_memory_20261011T095554571762Z --output NEW_REPORT_DIRECTORY`. INPUTS_SHA256.json identifies every source summary; PLOTS.json records filters, units and transformations. FINAL_AUDIT.md and COMMANDS.md retain actual validation/build/experiment commands and failures. Raw CSV remains immutable; derived outputs use new directories.

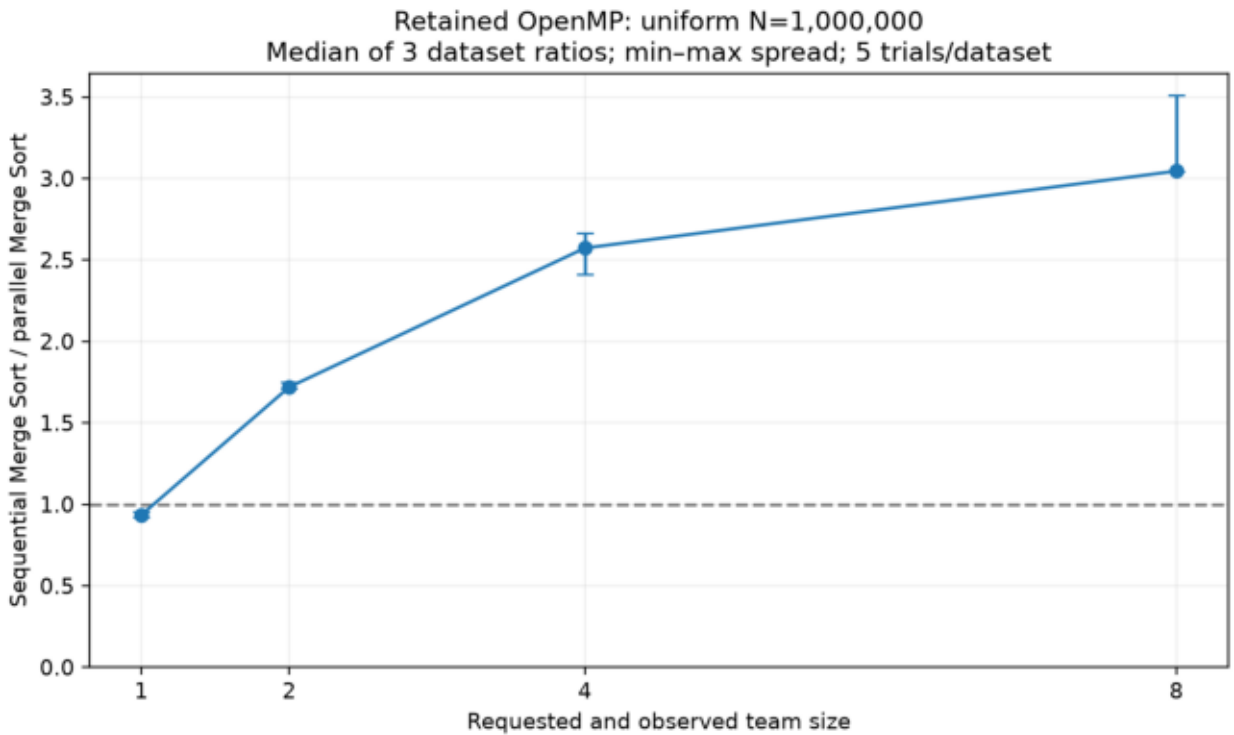
Primary references: [PDQSort](https://github.com/orlp/pdqsort); [cpp-TimSort](https://github.com/timsort/cpp-TimSort); [Kraska et al., SIGMOD2018](https://arxiv.org/abs/1712.01208v3); [Mankowitz et al., Nature2023](https://www.nature.com/articles/s41586-023-06004-9); [official AlphaDev archive](https://github.com/google-deepmind/alphadev).

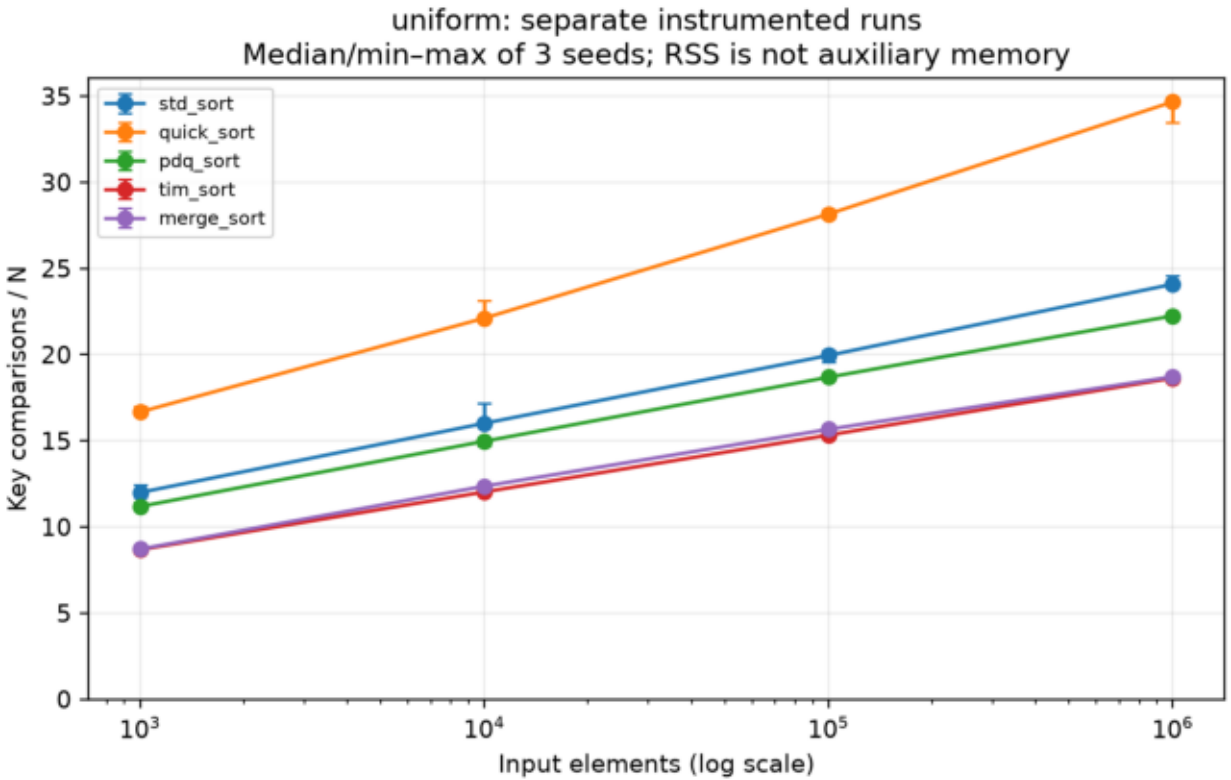

Retained 1M uniform study
Median of 3 seed medians; bars show cross-seed min–max; 5 trials/seed
Sort latency (ms)
0
20
40
60
80
std_sort
quick_sort
pdq_sort
tim_sort
merge_sort
radix_sort

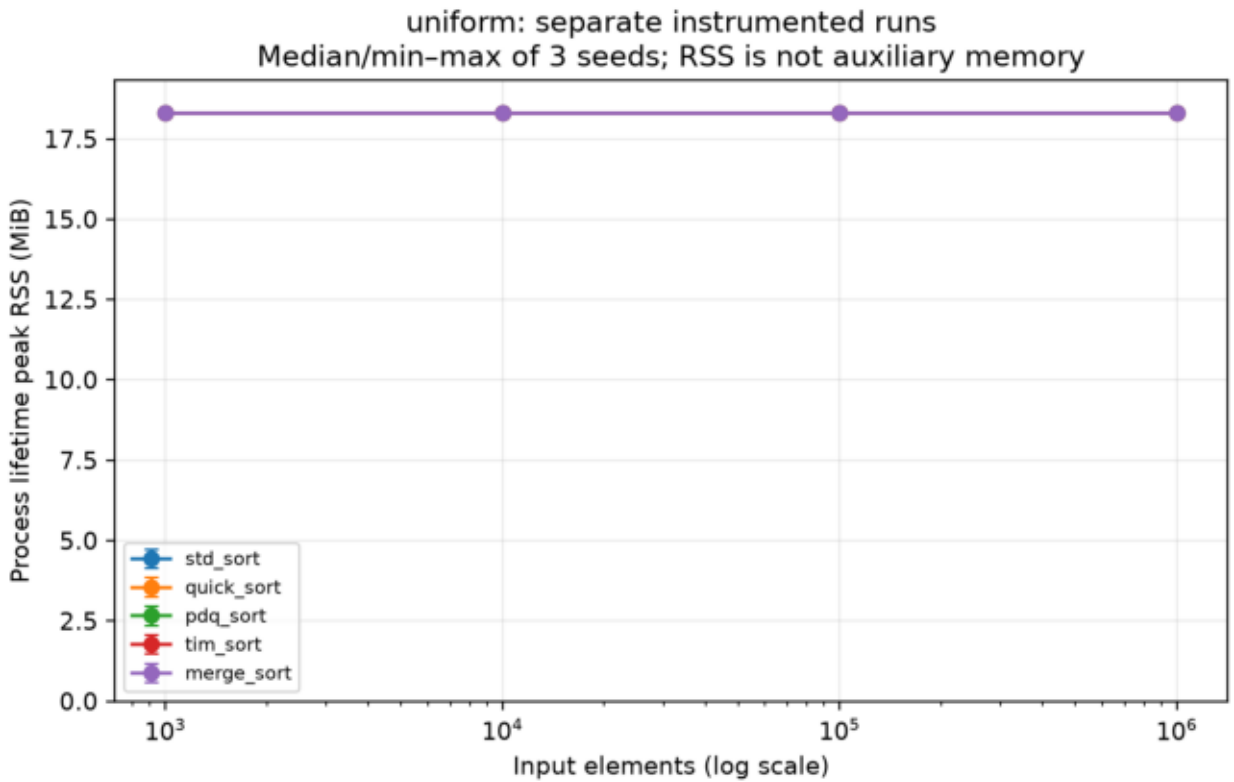

Uniform scaling: observed CPU study medians
1M/10M: 3 seeds; 100M: 1 seed; 5 trials/dataset; no extrapolation
merge_sort
pdq_sort
quick_sort
radix_sort
std_sort
tim_sort
Median sort latency (seconds, log scale)
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{6}
10^{7}
10^{8}
Input keys (log scale)


Retained search, uniform N=1000000, 50% hits
Dataset medians, then equal-weight distinct inputs; 1000 queries/batch
Normalized batch average (ns/query, log scale)
10^5
10^4
10^3
10^2
binary_search
exponential_search
linear_search


Retained structures, uniform N=1000000, 50% hits
Dataset medians, then equal-weight distinct inputs; 1000 queries/batch
Normalized batch average (ns/query, log scale)
10³
6 × 10²
4 × 10²
3 × 10²
2 × 10²
btree_m10
btree_m16
btree_m32
btree_m64
skip_list


Retained OpenMP: uniform N=1,000,000
Median of 3 dataset ratios; min–max spread; 5 trials/dataset
Sequential Merge Sort / parallel Merge Sort
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
1
2
4
8
Requested and observed team size


uniform: separate instrumented runs
Median/min–max of 3 seeds; RSS is not auxiliary memory
std_sort
quick_sort
pdq_sort
tim_sort
merge_sort
Key comparisons / N
Input elements (log scale)
0
5
10
15
20
25
30
35
10^3
10^4
10^5
10^6


uniform: separate instrumented runs
Median/min–max of 3 seeds; RSS is not auxiliary memory
Process lifetime peak RSS (MiB)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
std_sort
quick_sort
pdq_sort
tim_sort
merge_sort
10^3
10^4
10^5
10^6
Input elements (log scale)